

Introduction to Deep Learning

Session 9: Convolutional Neural Networks

May 2026

Magda Gregorová - magda.gregorova@thws.de



Licensed according to CC-BY-SA 4.0

Lecture Overview

1. Why Not MLPs for Images?

2. Convolution Operation

3. CNN Architecture

4. Classic Architectures

5. Modern Details

6. Summary

Images in Computers

Image: 3D tensor of shape $C \times H \times W$

- Each entry: pixel intensity, integer 0–255 or float 0–1,
- 2^8 possible values ~ 8 bits
- **Grayscale** ($C = 1$): single channel, 0 = black, 255 = white
- **RGB** ($C = 3$): red, green, blue — standard cameras, screens

Other formats:

- **RGBA** ($C = 4$): RGB + α transparency
- **CMYK** ($C = 4$): print format
- **Medical/hyperspectral**: many channels, often 16-bit depth

DL mostly **RGB and grayscale** — other formats typically converted before use

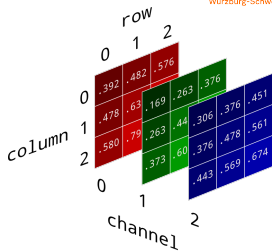


Image sizes — from benchmarks to real world:

Source	Size	Channels	Pixels
MNIST	28×28	1	784
ImageNet	224×224	3	150 528
Phone camera	4000×3000	3	36 000 000
Full HD screen	1920×1080	3	6 220 800
Medical (CT slice)	512×512	1	262 144

MLP approach: flatten image to vector $\mathbf{x} \in \mathbb{R}^{H \cdot W \cdot C}$, apply fully connected layers

ImageNet: first hidden from $\mathbb{R}^{150\,528} \rightarrow \mathbb{R}^{1024} \Rightarrow \approx 154\text{M}$ parameters — one layer only!

How to handle high-dimensional data efficiently?

How to Look at Images

Human vision:

- Looks for local patterns — edges, corners, textures
- Same pattern recognised anywhere in the visual field
- Simple features combine into complex ones

Technical equivalent:

- **Local connectivity** — connect to small patches only
- **Parameter sharing** — same filter at every location
- **Hierarchical features** — edges → shapes → objects

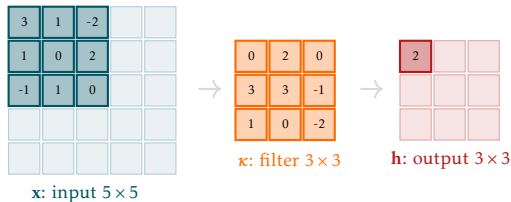
Three key properties: **locality, translation invariance, parameter efficiency**

These define the **convolutional layer**

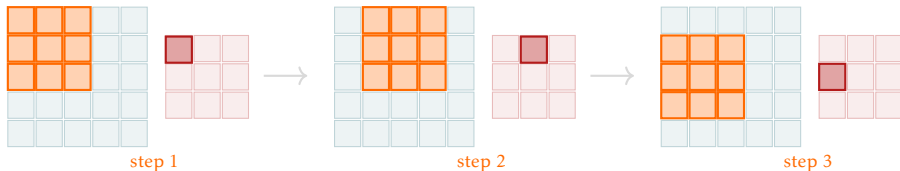
Convolution Operation

Convolution — The Operation

Single step: dot product of local patch and filter $\rightarrow$ one output value



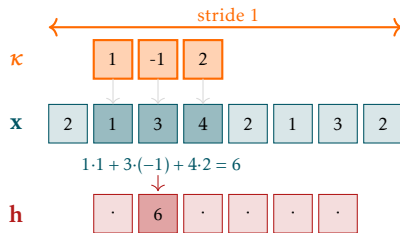
Full operation: same filter slides across input — each position produces one output value



All 3 requirements: local connectivity, parameter sharing, translation invariance

1D Convolution

Filter $\kappa \in \mathbb{R}^k$ slides over signal $\mathbf{x} \in \mathbb{R}^n$ — at each position i , dot product with local patch



$$(\mathbf{x} * \kappa)[i] = \sum_{j=0}^{k-1} \mathbf{x}[i+j] \kappa[j]$$

- i — output position; j — index within filter
- Dot product of filter with local patch

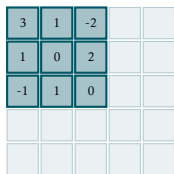
Example: at position $i = 1$:

$$1 \cdot 1 + 3 \cdot (-1) + 4 \cdot 2 = 6$$

2D Convolution

Extend to 2D: filter $\kappa \in \mathbb{R}^{k \times k}$ slides over image $\mathbf{X} \in \mathbb{R}^{H \times W}$:

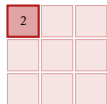
$$(\mathbf{X} * \kappa)[i, j] = \sum_{p=0}^{k-1} \sum_{q=0}^{k-1} \mathbf{X}[i + p, j + q] \kappa[p, q]$$



$\mathbf{x}$: input 5×5



κ : filter 3×3



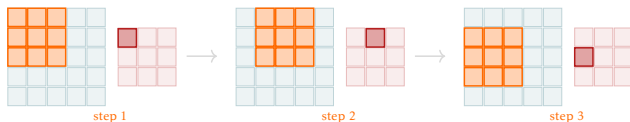
$\mathbf{h}$: output 3×3

- (i, j) — output position
- (p, q) — index within filter
- Two sums: over rows and columns of filter

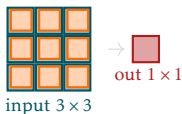
Output size: $(H - k + 1) \times (W - k + 1)$

Padding

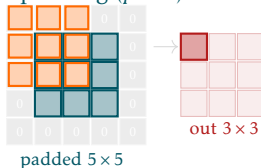
Problem: convolution shrinks spatial dimensions — each layer loses $k - 1$ pixels



no padding



same padding ($p = 1$)



Valid padding ($p = 0$)

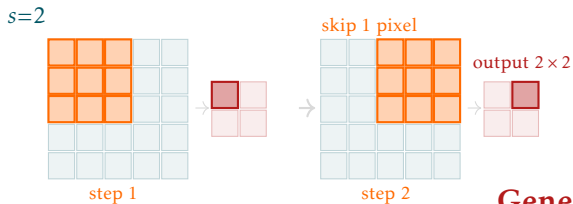
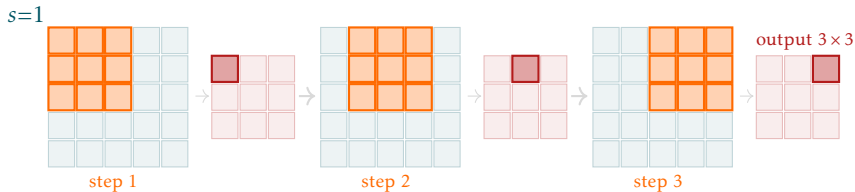
- No zeros added
- Output shrinks: $(H - k + 1) \times (W - k + 1)$
- Spatial information lost at borders

Same padding ($p = (k - 1)/2$)

- Zeros added around border
- Output same size as input
- Standard choice in most architectures

Stride

Stride s : filter κ moves s pixels at each step instead of 1



Stride 1: κ moves one step a time

Stride 2: κ skips one - output halved

General output size:
$$\left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1$$

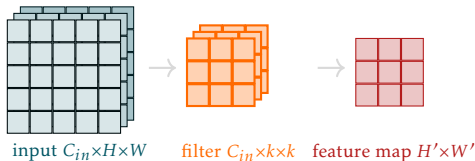
Multiple Input Channels

RGB input: C_{in} channels — filter must have same depth C_{in}

Operation: filter has C_{in} slices — each slice convolves with one input channel

$$(\mathbf{X} * \kappa)[i, j] = \underbrace{\sum_{c=0}^{C_{in}-1} \sum_{p=0}^{k-1} \sum_{q=0}^{k-1} \mathbf{X}[c, i+p, j+q] \kappa[c, p, q]}_{\text{conv with channel } c}$$

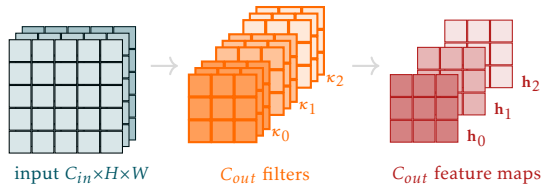
- Inner double sum: convolution of filter slice c with input channel c
- Outer sum over c : results summed across all channels
- One filter $\rightarrow$ one output feature map



Multiple Output Channels

C_{out} **filters**: each filter independently applied to the full input volume

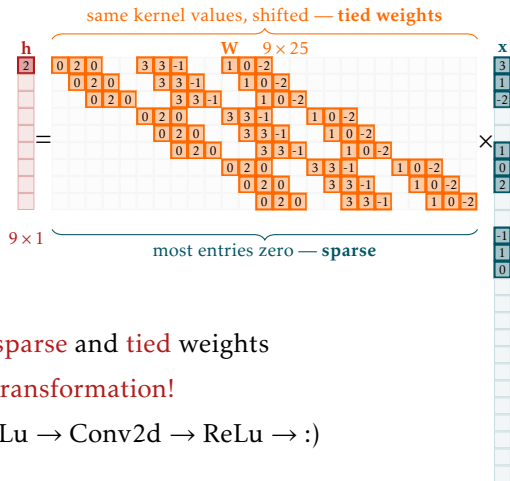
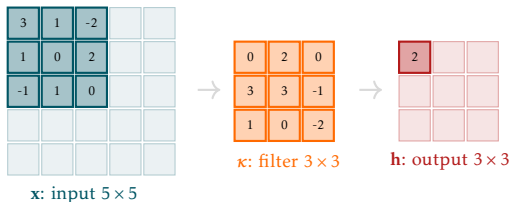
$$\mathbf{h}_f[i, j] = \sum_{c=0}^{C_{in}-1} \sum_{p=0}^{k-1} \sum_{q=0}^{k-1} \mathbf{X}[c, i+p, j+q] \kappa_f[c, p, q] \quad f = 0, \dots, C_{out} - 1$$



- Each filter κ_f produces one feature map $\mathbf{h}_f$ independently
- Output tensor: $C_{out} \times H' \times W'$
- Parameters: $C_{out} \times C_{in} \times k \times k + C_{out}$ (with bias)

Convolution as Matrix Multiplication

Flatten input and output — convolution becomes matrix multiplication



Convolution = FC layer with **sparse** and **tied** weights

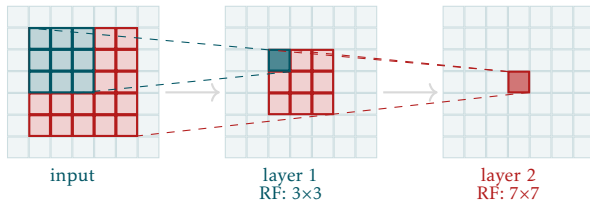
Convolution - **linear transformation!**

Conv2d $\rightarrow$ ReLu $\rightarrow$ Conv2d $\rightarrow$ ReLu $\rightarrow$ Conv2d $\rightarrow$ ReLu $\rightarrow$:)

CNN Architecture

Receptive Field

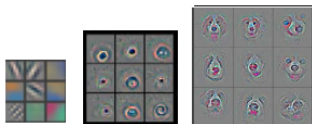
Receptive field: region of the input that influences a given neuron's output



- Each 3×3 conv layer adds 2 pixels to the receptive field in each dimension
- L layers of 3×3 filters: receptive field = $(2L + 1) \times (2L + 1)$
- Deep networks see large regions despite small local filters
- Early layers: local features (edges) — later layers: global patterns (objects)

What Do CNNs Learn?

Zeiler & Fergus (2014) visualised features learned at each layer of a CNN trained on ImageNet:



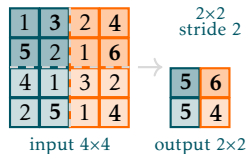
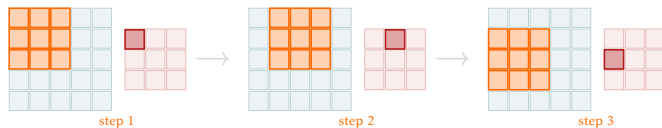
- **Layer 1–2:** edges, colours, orientations
- **Layer 3:** textures, repeated patterns
- **Layer 4–5:** object parts, faces, complex structures

Hierarchical features — simple features combine into complex ones through depth

Pooling

Problem: convs features spatially redundant - overlapping receptive field → nearby cells highly correlated

Solution pooling: fixed non-trainable kernel slides through; non-overlapping regions (stride = pool size); no channel interaction (independent per channel)



Max pooling (most common):

$$y[i, j] = \max_{p, q \in \mathcal{R}_{ij}} x[p, q]$$

Retains strongest activation in each region

Average pooling:

$$y[i, j] = \frac{1}{|\mathcal{R}_{ij}|} \sum_{p, q \in \mathcal{R}_{ij}} x[p, q]$$

Effect: $C \times H \times W \xrightarrow{2 \times 2 \text{ pool}} C \times H/2 \times W/2$ — channel dimension unchanged

Problem: flattening feature maps before FC layers is expensive

$C \times H \times W$ flattened $\rightarrow C \cdot H \cdot W$ inputs to FC layer — huge parameter count (VGG: ~100M parameters in FC layers alone)

Global average pooling (GAP): average each feature map to a single value

$$y_c = \frac{1}{H \cdot W} \sum_{i=1}^H \sum_{j=1}^W x_c[i, j] \quad c = 1, \dots, C$$

$C \times H \times W \xrightarrow{\text{GAP}} C \times 1 \times 1$ — one value per feature map

Intuition: each feature map represents one concept — its average activation measures how present that concept is in the image

Benefits:

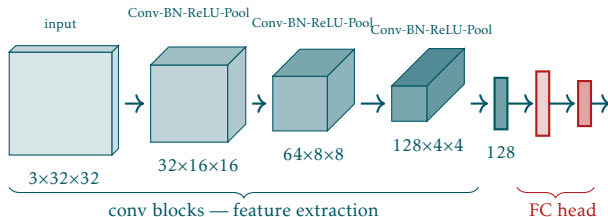
- Eliminates large FC layers — dramatic parameter reduction
- Works with any input image size — no fixed spatial dimension required

Standard pattern: Conv $\rightarrow$ BatchNorm $\rightarrow$ ReLU $\rightarrow$ (Pool)

- **Conv:** extract features via learned filters
- **BatchNorm:** stabilise activations — essential for deep CNNs
- **ReLU:** non-linearity — enables complex feature combinations
- **Pool:** spatial downsampling — reduce resolution, increase receptive field

Full CNN Architecture

Two stages: feature extraction (conv blocks) → classification (FC head)



Conv blocks:

- Increasing channels: $3 \rightarrow 32 \rightarrow 64 \rightarrow 128$
- Decreasing spatial size via pooling
- Learn hierarchical features

FC head:

- Flatten spatial features
- One or more FC layers
- Final softmax for classification

LeNet-5 (LeCun et al., 1998)

First successful CNN — handwritten digit recognition (MNIST)

- 2 conv layers + 2 FC layers
- 28×28 input $\rightarrow$ 10 class output
- ~60k parameters — tiny by modern standards
- Used sigmoid/tanh activations, average pooling

Significance:

- Demonstrated that learned features outperform hand-crafted ones
- Introduced the conv $\rightarrow$ pool $\rightarrow$ conv $\rightarrow$ pool $\rightarrow$ FC pattern
- Deployed commercially for cheque reading

Limitation: too shallow and small for complex tasks — needed more compute

AlexNet (Krizhevsky et al., 2012)

ImageNet moment — won ILSVRC 2012 with 15.3% top-5 error vs 26.2% runner-up

- 5 conv layers + 3 FC layers, ~60M parameters
- Key innovations:
 - **ReLU activations** instead of sigmoid/tanh — faster training
 - **Dropout** in FC layers — regularisation
 - **Data augmentation** — random crops, horizontal flips
 - **GPU training** — two GTX 580 GPUs
 - Local response normalisation (since superseded by BatchNorm)

Impact: launched the modern deep learning era — proved deep CNNs work at scale

VGG (Simonyan & Zisserman, 2014)

Simplicity through depth — all 3×3 convolutions, increasing depth

- VGG-16: 13 conv layers + 3 FC layers, ~138M parameters
- Only 3×3 filters throughout — simplicity as a design principle
- Depth: 16–19 layers

Key insight: two 3×3 conv layers have same receptive field as one 5×5 but:

- Fewer parameters: $2 \times 3^2 = 18$ vs $5^2 = 25$
- More non-linearities — more expressive

Limitation: 3 large FC layers contain most parameters (~100M) — memory intensive

Legacy: VGG features widely used for transfer learning; clean design still instructive

ResNet (He et al., 2016)

Problem: deeper networks were performing **worse** than shallower ones

Degradation problem: adding more layers hurts training accuracy — not overfitting, but optimisation failure

Key idea: residual connections (skip connections)

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{\mathbf{W}_i\}) + \mathbf{x}$$

- Instead of learning $\mathbf{y} = \mathcal{F}(\mathbf{x})$, learn the **residual** $\mathcal{F}(\mathbf{x}) = \mathbf{y} - \mathbf{x}$
- If optimal solution is identity, easier to learn $\mathcal{F}(\mathbf{x}) = 0$ than $\mathcal{F}(\mathbf{x}) = \mathbf{x}$
- Gradient flows directly through skip connection — no vanishing gradients

Result: successfully trained networks with 50, 101, 152 layers

Residual Block

Basic residual block:

$$\mathbf{y} = \text{ReLU}(\mathcal{F}(\mathbf{x}) + \mathbf{x})$$

$\mathcal{F}(\mathbf{x})$: two Conv-BN-ReLU layers

Bottleneck block (ResNet-50+):

- 1×1 conv: reduce channels
- 3×3 conv: spatial processing
- 1×1 conv: restore channels

Fewer parameters than basic block at same depth

Projection shortcut: when dimensions change,
use 1×1 conv on skip connection to match

1 × 1 Convolutions

1 × 1 **conv**: applies a learned linear combination across channels at each spatial location

$$y_c[i, j] = \sum_{c'=1}^{C_{\text{in}}} w_{c,c'} x_{c'}[i, j] + b_c$$

Equivalent to: FC layer applied independently at each spatial location

Uses:

- **Channel reduction/expansion** — change C_{in} to C_{out} cheaply (bottleneck blocks)
- **Non-linearity injection** — add ReLU between channels without spatial processing
- **Network-in-network** — increase model capacity without larger filters
- **Projection shortcuts** — match dimensions in residual connections

Depthwise Separable Convolutions

Standard conv: $C_{\text{out}} \times C_{\text{in}} \times k \times k$ parameters

Depthwise separable conv: split into two steps

Depthwise conv:

- One filter per input channel
- Spatial processing only
- $C_{\text{in}} \times k \times k$ parameters

Pointwise conv (1×1):

- Mix channels
- $C_{\text{out}} \times C_{\text{in}}$ parameters

Parameter reduction: factor of $\approx k^2$ — for 3×3 : $\sim 9\times$ fewer parameters

Used in: MobileNet, EfficientNet — efficient architectures for mobile/edge deployment

BatchNorm2d: normalises over batch **and** spatial dimensions for each channel

$$\mu_c = \frac{1}{N \cdot H \cdot W} \sum_{n,i,j} x_c^{(n)}[i,j], \quad \sigma_c^2 = \frac{1}{N \cdot H \cdot W} \sum_{n,i,j} (x_c^{(n)}[i,j] - \mu_c)^2$$

- One mean and variance per **channel** — not per spatial location
- Learnable γ_c and β_c per channel

Placement: Conv $\rightarrow$ **BN** $\rightarrow$ ReLU (standard) or Conv $\rightarrow$ ReLU $\rightarrow$ BN (less common)

Effect in CNNs:

- Essential for training deep CNNs — without it, networks beyond ~ 10 layers are very hard to train
- Allows higher learning rates
- Reduces need for careful initialisation

Standard dropout on FC layers works well in CNNs too

Problem with dropout on conv layers:

- Adjacent pixels are highly correlated
- Dropping individual activations does not create enough independence
- Information easily propagated through neighbouring non-dropped activations

SpatialDropout2d: drop entire feature maps (channels) rather than individual activations

- Removes all spatial locations of a channel simultaneously
- Forces network to use diverse feature maps
- More effective regularisation for conv layers

In practice: BatchNorm often sufficient regularisation in conv layers — dropout mainly used in FC head

Why CNNs:

- Local connectivity
- Parameter sharing
- Translation invariance

Building blocks:

- Conv: feature extraction
- BatchNorm + ReLU: stability + non-linearity
- Pooling / stride: downsampling
- FC head: classification

Key architectures:

- LeNet → AlexNet → VGG → ResNet
- Residual connections: key to deep networks

Modern details:

- 1×1 conv: channel mixing
- Depthwise separable: efficiency
- BatchNorm2d: per-channel normalisation
- SpatialDropout: conv regularisation

In PyTorch:

- `nn.Conv2d`
- `nn.MaxPool2d`
- `nn.BatchNorm2d`
- `nn.AdaptiveAvgPool2d`

Next: L10 — Sequence Models & RNNs